

Reviewer Pack — Minimal Rank of Quasi-Parseval Spaces and Communication Comp...

Entry 9e569bba8d67 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 04:50:13 UTC

Minimal Rank of Quasi-Parseval Spaces and Communication Complexity Rank

Entry ID: 9e569bba8d67 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 04:50:13 UTC

1. Conjecture

Field A (mathematical branch): Functional Analysis (Quasi-Parseval Spaces) **Field B** (complexity object): Communication Complexity

Statement:

For every n -ary communication protocol P , the minimal rank of its associated quasi-Parseval space $Q(P)$ is $O(\log(n))$ and upper-bounded by the communication complexity rank $r(P)$, such that $\log(n) \leq r(Q(P)) \leq r(P)$.

Rationale (proposer's reasoning):

Quasi-Parseval spaces provide a mathematical framework for understanding the structure of functions in signal processing and quantum information theory. Their minimal rank may offer insights into the inherent complexity of communication protocols, potentially leading to new methods for proving lower bounds on communication complexity.

Taxonomy category: communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 07e4493e2d30075b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated n -ary communication protocols P with varying communication complexity ranks $r(P)$, the associated quasi-Parseval space $Q(P)$ has a minimal rank $m(Q(P))$ that satisfies $\log(n) \leq m(Q(P)) \leq r(P)$. The conjecture is falsified if any P has $m(Q(P)) > r(P)$ or $m(Q(P)) < \log(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_protocol(n):
        # Generate a random n-ary communication protocol
        return [random.randint(0, 1) for _ in range(n)]

    def compute_quasi_parseval_space(protocol):
        # Compute the quasi-Parseval space Q(P) associated with the protocol
        qps = []
        for p in protocol:
            if p == 0:
                qps.append([1, -1])
            else:
                qps.append([-1, 1])
        return qps

    def min_rank(matrix):
        # Compute the minimal rank of a matrix
        n = len(matrix)
        m = len(matrix[0])
        rank = 0
        for i in range(n):
            if all(matrix[i][j] == 0 for j in range(m)):
                continue
            rank += 1
            for j in range(m):
                matrix[i][j] /= matrix[i][j]
            for k in range(n):
                if k != i and any(matrix[k][j] != 0 for j in range(m)):
                    for j in range(m):
                        matrix[k][j] -= matrix[i][j] * matrix[k][j]
        return rank
```

```

def communication_complexity_rank(protocol):
    # Compute the communication complexity rank  $r(P)$ 
    n = len(protocol)
    rank = 0
    for i in range(n):
        if protocol[i] == 0:
            rank += 1
        else:
            rank -= 1
    return abs(rank) // 2

n_values = [5, 10, 15, 20, 30, 40]
total_instances = 0
min_ranks = []
r_Ps = []

for n in n_values:
    for _ in range(5): # Test each n with 5 different instances
        protocol = generate_protocol(n)
        qps = compute_quasi_parseval_space(protocol)
        min_rank_value = min_rank(qps)
        r_P_value = communication_complexity_rank(protocol)

        total_instances += 1
        min_ranks.append(min_rank_value)
        r_Ps.append(r_P_value)

mean_min_rank = sum(min_ranks) / len(min_ranks)
mean_r_P = sum(r_Ps) / len(r_Ps)

conjecture_holds = all(log(n) <= m <= r for n, m, r in zip(n_values, min_ranks, r_Ps))
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "min_rank",
    "metric_value": mean_min_rank,
    "instances_tested": total_instances,
    "n_max": max(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

```

```

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=unsupported_operation")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3b75554f.py", line 100, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3b75554f.py", line 82, in run_trial
    conjecture_holds = all(log(n) <= m <= r for n, m, r in zip(n_values, min_ranks, r_Ps))
                          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_3b75554f.py", line 82, in <genexpr>
    conjecture_holds = all(log(n) <= m <= r for n, m, r in zip(n_values, min_ranks, r_Ps))
                          ^^
NameError: name 'log' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to an undefined 'log' function, preventing the evaluation of the conjecture. | next: Investigate and fix the error in the test code that caused it to crash. Once fixed, re-run the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15570
2	propose	ollama_remote	glm4:latest	0	0	16785
3	propose	ollama_remote	glm4:latest	0	0	12079
4	preregistration	ollama_remote	glm4:latest	0	0	9951
5	novelty	ollama_remote	glm4:latest	0	0	8555
6	novelty	ollama_remote	glm4:latest	0	0	8452

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13867
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20287
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21079
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17997
11	judge	ollama_remote	glm4:latest	0	0	10157

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 154778 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/9e569bba8d67.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/9e569bba8d67.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/9e569bba8d67.tar.gz* (if generated)